

Ch1.0 Outline

(1) Mathematical Preliminaries & Notation (2) Three Basic Concepts
(3) Some Applications (*omitted*).
Core tools: **Sets, Functions, Relations, Graphs, Proof Techniques**; then **Languages, Grammars, Automata (Chap. 2)**.

1.1 Sets

A **set** is a collection of elements. $A = \{1, 2, 3\}$, $B = \{\text{train}, \text{bus}, \text{bicycle}, \text{airplane}\}$.

$$1 \in A, \quad \text{ship} \notin B$$

Representations: explicit/finite $C = \{a, \dots, k\}$; infinite $S = \{2, 4, 6, \dots\} = \{j : j > 0, j = 2k, k > 0\}$; **set-builder** $S = \{j : j \text{ nonneg. and even}\}$. **Universal set** U = all possible elements (context set), e.g. $U = \{1, \dots, 10\}$.

Set Operations ▶

$A = \{1, 2, 3\}$, $B = \{2, 3, 4, 5\}$:

- Union** $A \cup B = \{1, 2, 3, 4, 5\}$
- Intersection** $A \cap B = \{2, 3\}$
- Difference** $A - B = \{1\}$, $B - A = \{4, 5\}$
- Complement** $\bar{A} = U - A$; e.g. $U = \{1, \dots, 7\}$, $A = \{1, 2, 3\} \Rightarrow \bar{A} = \{4, 5, 6, 7\}$

$\overline{\bar{A}} = A$; complementary sets: $\overline{\{\text{even ints}\}} = \{\text{odd ints}\}$ under $\mathbb{Z}$.

DeMorgan's Laws ▶

$$\overline{A \cup B} = \bar{A} \cap \bar{B}, \quad \overline{A \cap B} = \bar{A} \cup \bar{B}$$

(Note: slides write $\overline{A \cup B} = \bar{A} \bar{B}$ compactly.)

Empty / Null Set ▶

$\emptyset = \{\}$ = set with no elements.

$$S \cup \emptyset = S, \quad S \cap \emptyset = \emptyset, \quad S - \emptyset = S, \quad \emptyset - S = \emptyset$$

($\emptyset \cap U = \emptyset$, union with U gives U .)

Key: $\emptyset \neq \{\lambda\}$! Set size $|\{\}\| = |\emptyset| = 0$; but $|\{\lambda\}| = 1$. Here λ = empty string (a set element).

Subset ▶

$A = \{1, 2, 3\}$, $B = \{1, 2, 3, 4, 5\}$.

- Subset** $A \subseteq B$: every element of A is also in B .
- Proper subset** $A \subset B$: $A \subseteq B$ but B has an element not in A .

Disjoint & Cardinality ▶

Disjoint: $A \cap B = \emptyset$. e.g. $A = \{1, 2, 3\}$, $B = \{5, 6\}$.

Cardinality $|A|$ for finite A : $A = \{2, 5, 7\} \Rightarrow |A| = 3$ (set size). $|\emptyset| = 0$.

Powerset ▶

Powerset $2^S = \mathcal{P}(S)$ = set of *all* subsets.

$$S = \{a, b, c\} : 2^S = \{\emptyset, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$$

Key: If S finite, $|2^S| = 2^{|S|}$ (here $8 = 2^3$).

Cartesian Product ▶

$A \times B = \{(x, y) : x \in A, y \in B\}$.

Ex) (Ex 1.2) $A = \{2, 4\}$, $B = \{2, 3, 5, 6\}$:

$$A \times B = \{(2, 2), (2, 3), (2, 5), (2, 6), (4, 2), (4, 3), (4, 5), (4, 6)\}$$

$|A \times B| = |A| \cdot |B|$. Order matters: $(4, 2) \in A \times B$, but $(2, 4) \notin A \times B$.

Partition ▶

$S_1, \dots, S_n \subseteq S$ form a **partition** of S iff:

- mutually disjoint: $S_i \cap S_j = \emptyset$ ($i \neq j$);
- $S_1 \cup S_2 \cup \dots \cup S_n = S$;
- none of the S_i is empty.

1.2 Functions

A **function** assigns to each element of one set a *unique* element of another set: $f : A \rightarrow B$.

- A = **domain**, B = **range**; $f(1) = a$.
- If A = domain, f is a **total function**; otherwise f is a **partial function**.

1.3 Asymptotic Analysis

Big-Oh (upper bound) $\mathcal{O}$: $f(n) = \mathcal{O}(g(n))$ if $\exists c > 0, n_0 > 0$ s.t. $0 \leq f(n) \leq c g(n)$ for all $n \geq n_0$.

Intuition: f “ $<$ ” g ignoring constant multiples and small n .

Ratio test: $f(n) = \mathcal{O}(g(n))$ implies $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c$ for some $c \geq 0$ (if the limit exists).

Big-Omega (lower bound) Ω : $f(n) = \Omega(g(n))$ if $\exists c > 0, n_0 > 0$ s.t. $0 \leq c g(n) \leq f(n)$ for all $n \geq n_0$.

Intuition: f “ $=$ ” g ignoring constants and small n .

Ratio test: $f(n) = \Omega(g(n))$ implies $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c$ with $c > 0$ (or diverges to ∞), if the limit exists.

Big-Theta Θ : $f = \Theta(g)$ iff $f = \mathcal{O}(g)$ and $f = \Omega(g)$ (tight bound).

- $3n^2 + n = \mathcal{O}(n^2)$? **Yes** / $\mathcal{O}(n)$? **No** / $\mathcal{O}(n^3)$? **Yes**
- $3n^2 + n = \Omega(n^2)$? **Yes** / $\Omega(n)$? **Yes** / $\Omega(n^3)$? **No**

Ex) (Ex 1.3) $f(n) = 2n^2 + 3n$, $g(n) = n^3$, $h(n) = 10n^2 + 100$:

$$f = \mathcal{O}(g), \quad g = \Omega(h), \quad \underbrace{f = \Theta(h)}_{2n^2+3n \text{ vs } 10n^2+100}$$

1.4 Relations

More general than functions:

- Function:** each domain element has *exactly one* associated range element.
- Relation:** there may be *several* such elements.

$R = \{(x_1, y_1), (x_2, y_2), \dots\}$, write $x_i R y_i$.

E.g. $R = \text{'>'}: 2 > 1, 3 > 2, 3 > 1$.

Equivalence Relations ($\equiv$) ▶

R is an equivalence relation iff:

- Reflexive:** $x R x$
- Symmetric:** $x R y \Rightarrow y R x$
- Transitive:** $x R y$ and $y R z \Rightarrow x R z$

Ex) (Ex 1.4) On nonneg. integers define $x \equiv y$ iff $x \bmod 3 = y \bmod 3$. Then $2 \equiv 5$, $12 \equiv 0$, $0 \equiv 36$; it is an equivalence relation (reflexive, symmetric, transitive).

Example ‘ $=$ ’ is an equivalence: $x = x$; $x = y \Rightarrow y = x$; $x = y \wedge y = z \Rightarrow x = z$.

1.5 Graphs

A **directed graph** (digraph) $G = (V, E)$:

- Nodes (vertices)** $V = \{a, b, c, d, e\}$
- Edges** $E = \{(a, b), (b, c), (b, e), (c, a), (c, e), (d, c), (e, b), (e, d)\}$

Labeled graph: edges carry weights/labels.

Walk / Path / Cycle ▶

- Walk:** sequence of *adjacent* edges, e.g. $(e, d), (d, c), (c, a)$.
- Path:** a walk where *no edge is repeated*.
- Simple path:** no *node* is repeated.
- Cycle:** a walk from a node (**base**) back to itself.
- Simple cycle:** only the base node is repeated.
- Loop:** an edge from a vertex to itself.

E.g. in the graph: $(v_1, v_3), (v_3, v_2)$ = simple path $v_1 \rightarrow v_2$; $(v_1, v_3), (v_3, v_3), (v_3, v_1)$ = a cycle (*not simple*); loop on v_3 .

Euler Tour vs Hamiltonian Cycle ▶

- Euler tour:** a cycle that contains *each edge once*.
- Hamiltonian cycle:** a *simple* cycle that contains *all nodes*.

Finding All Simple Paths ▶

From an **origin**, grow paths edge-by-edge, never repeating a node.

Ex) Starting at c : Step1 $(c, a), (c, e)$; Step2 add $(c, a)(a, b), (c, e)(e, b), (c, e)(e, d)$; Step3 extend to $(c, a)(a, b)(b, e)$; Step4 $(c, a)(a, b)(b, e)(e, d)$. (Depth-first enumeration.)

Trees ▶

Tree: connected graph with *no cycles*. Terminology: **root** (top), **parent/child**, **leaf** (no children), **level** (root = Level 0), **height** = longest root–leaf path.

1.6 Proof Techniques

Three families: **Direct/Constructive**, **Induction**, **Contradiction**.

Direct / Constructive Proof ►

- Form: “If X , then Y .”
- Assume X true, show directly that Y is true. (e.g. X = “it rains”, Y = “sidewalk will be wet”.)

Ex) Claim: if a, b odd then ab odd. **Proof (constructive):** let $a = 2x + 1, b = 2y + 1$; then

$$ab = 4xy + 2x + 2y + 1 = 2(2xy + x + y) + 1$$

so $ab = 2z + 1$ with $z = 2xy + x + y$ — an odd integer. Not only proves such z *exists*, it *constructs* the recipe.

Proof by Induction ►

Statements $P_1, P_2, P_3, \dots$. If

- for some b , $P_1, \dots, P_b$ are true (**basis**); and
- for any $k \geq b$, $P_1, \dots, P_k$ imply P_{k+1} (**step**),

then every P_i is true.

- Inductive basis:** find $P_1, \dots, P_b$ true.
- Inductive hypothesis:** assume $P_1, \dots, P_k$ true for $k \geq b$.
- Inductive step:** show P_{k+1} .

Ex) (Ex 1.5) Thm: a binary tree of height n has at most 2^n leaves. Let $L(i)$ = max leaves of a subtree at height i . Show $L(i) \leq 2^i$. **Basis:** $L(0) = 1$ (root). **Hyp:** $L(i) \leq 2^i$ for $i = 0, \dots, k$. **Step:** each level- k leaf adds ≤ 2 children:

$$L(k+1) \leq 2L(k) \leq 2 \cdot 2^k = 2^{k+1}.$$

Remark: recursion is different: e.g. $f(n) = f(n-1) + f(n-2)$, $f(0) = f(1) = 1$.

Ex) (Ex 1.6) n mutually intersecting lines split the plane into regions. Let $A(n)$ = number of regions. 1 line $\rightarrow$ 2, 2 lines $\rightarrow$ 4, 3 lines $\rightarrow$ 7. Recurrence

$$A(n+1) = A(n) + n + 1, \quad A(1) = 2$$

$$A(n) = \frac{n(n+1)}{2} + 1 \quad (\text{solve: } A(2) = 4, A(3) = 7, A(4) = 11)$$

Proof by Contradiction ►

To prove P : assume P is *false*; derive an incorrect conclusion; therefore P must be true.

Ex) (Ex 1.7) Thm: $\sqrt{2}$ is not rational.

Assume $\sqrt{2} = n/m$ with n, m coprime. Then $2m^2 = n^2 \Rightarrow n^2$ even $\Rightarrow n$ even, $n = 2k$. So $2m^2 = 4k^2 \Rightarrow m^2 = 2k^2 \Rightarrow m$ even, $m = 2p$. Then m, n share factor 2 — **contradiction** to coprimality.

2.1 Three Basic Concepts

Languages, Grammars, Automata (Chap. 2).

Alphabets & Strings ►

An **alphabet** Σ is a finite set of symbols, e.g. $\Sigma = \{a, b\}$. A **string** is a finite sequence of symbols; **language** = a *set of strings*.

$$u = ab, \quad v = bbb \, aaa, \quad w = abba, \quad \Sigma = \{a, b, c, \dots, z\}$$

String Operations ►

$$w = a_1 a_2 \dots a_n, \quad v = b_1 b_2 \dots b_m.$$

- Concatenation** $wv = a_1 \dots a_n b_1 \dots b_m$: $abba \cdot bbbaaa = abbabbbaaa$.
- Reverse** $w^R = a_n \dots a_2 a_1$: $(ababaaabbb)^R = bbbbaaababa$.
- Length** $|w| = n$: $|abba| = 4$, $|aa| = 2$, $|a| = 1$.
- Length of concatenation:** $|uv| = |u| + |v|$ (e.g. $3 + 5 = 8$).
- Empty string** λ (no letters): $|\lambda| = 0$, $\lambda w = w\lambda = w$, $\lambda abba = abba$.
- Substring:** consecutive characters, e.g. of $abbab$: $ab, abba, b, bbab$.
- Prefix / Suffix:** $w = uv$ (u prefix, v suffix) of $abbab$: prefixes $\lambda, a, ab, abb, abba, abbab$; suffixes $abbab, bbab, bab, ab, b, \lambda$.
- Power** $w^n = ww \dots w$ (n times): $(abba)^2 = abbaabba$; **def** $w^0 = \lambda$.

The Σ^* , Σ^+ Operations ►

Σ^* = all strings over Σ ; $\Sigma = \{a, b\} \Rightarrow \Sigma^* = \{\lambda, a, b, aa, ab, ba, bb, aaa, \dots\}$

$\Sigma^+ = \Sigma^* - \lambda = \{a, b, aa, ab, ba, bb, aaa, \dots\}$

Languages ►

A language is *any subset* of Σ^* . Examples:

- Finite: $\{\lambda\}$, $\{a, aa, aab\}$, $\{\lambda, abba, baba, aa, ab, aaaaaa\}$.
- Infinite: $L = \{a^n b^n : n \geq 0\}$; in: $ab, aabb, aaaaabbbb$; not-in: abb .

Operations on Languages ►

$X = \{a, ab, aaaa\}$, $Y = \{bb, ab\}$:

- $X \cup Y = \{a, ab, bb, aaaa\}$
- $X \cap Y = \{ab\}$
- $X - Y = \{a, aaaa\}$
- Complement** $\bar{L} = \Sigma^* - L$: $\overline{\{\lambda, ba\}} = \{\lambda, b, aa, ab, bb, aaa, \dots\}$
- Reverse** $L^R = \{w^R : w \in L\}$: $\{ab, aab, baba\}^R = \{ba, baa, abab\}$; $\{a^n b^n\}^R = \{b^n a^n\}$.
- Concatenation** $L_1 L_2 = \{xy : x \in L_1, y \in L_2\}$: $= \{ab, aaa, abb, abaa, bab, baaa\}$.
- Power** $L^n = LL \dots L$ (n times); $L^0 = \{\lambda\}$. $\{a, b\}^3 = \{aaa, aab, aba, abb, baa, bab, bba, bbb\}$.
- Star-closure (Kleene*)** $L^* = L^0 \cup L^1 \cup L^2 \cup \dots$ $\{a, bb\}^* = \{\lambda, a, bb, aa, abb, bba, bbbb, aaa, \dots\}$
- Positive closure** $L^+ = L^1 \cup L^2 \cup \dots = L^* - \{\lambda\}$

Ex) $L = \{a^n b^n\}$; $L^2 = \{a^n b^n a^m b^m\}$; note n, m are *unrelated* (e.g. $aabbbaabbb \in L^2$).

2.2 Grammars

A **grammar** describes how symbols are manipulated into valid strings; the study of rules governing language use.

Definition 1.1: A grammar is a 4-tuple $G = (V, T, S, P)$: V = finite set of **variables**; T = finite set of **terminals**; $S \in V$ = **start variable**; P = finite set of **production rules**.

English-analogy rules: Sentence = noun-phrase + verb-phrase; NP = Name | Article+noun; VP = Verb | Verb+PP; PP = Prep+NP.

E.g. “Joe runs from the car” is *valid* by successive substitution.

Derivations ►

Production $x \rightarrow y$, with $x \in (V \cup T)^+$, $y \in (V \cup T)^*$.

- One-step derive:** if $A \rightarrow \beta$ is a rule and $\alpha = \alpha_1 A \alpha_2$, then $\gamma = \alpha_1 \beta \alpha_2$; write $\alpha \Rightarrow \gamma$.
- Derives in 0+ steps:** $\alpha \xRightarrow{*} \gamma$.

Definition 1.2: $L(G) = \{w \in T^* : S \xRightarrow{*} w\}$ = language generated by G . A derivation $S \Rightarrow w_1 \Rightarrow \dots \Rightarrow w_n \Rightarrow w$; the $S, w_1, \dots, w_n$ are **sentential forms**.

Ex) (Ex 1.11) $G = (\{S\}, \{a, b\}, S, P)$ with $S \rightarrow aSb$, $S \rightarrow \lambda$.

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb \Rightarrow L(G) = \{a^n b^n : n \geq 0\}.$$

$aabb$ is a *sentence*; $aaSbb$ is a *sentential form*.

Ex) (Ex 1.12) Find G for $L = \{a^n b^{n+1} : n \geq 0\}$: generate one extra b .

$$G = (\{S, A\}, \{a, b\}, S, P) : S \rightarrow Ab, A \rightarrow aAb, A \rightarrow \lambda.$$

Ex) (Ex 1.13) $G: S \rightarrow SS \mid \lambda \mid aSb \mid bSa$ generates $L = \{w : n_a(w) = n_b(w)\}$.

Proof by induction: assume all $w \in L$ with $|w| \leq 2n$ derivable. Take $|w| = 2n+2$: if $w = aw_1b$ then $S \xRightarrow{*} w_1$, so $S \Rightarrow aSb \xRightarrow{*} aw_1b = w$ (similarly bSa); else $S \Rightarrow SS \xRightarrow{*} w_1 w_2 = w$.

Equivalent grammars (Ex 1.14): G_1, G_2 equivalent iff $L(G_1) = L(G_2)$. $G_1 : S \rightarrow aSb \mid \lambda \quad \equiv \quad G_2 : S \rightarrow aAb \mid \lambda, A \rightarrow aAb \mid \lambda$ (both generate $\{a^n b^n\}$).

CSCI 3130 Quiz 1 — generated from github.com/gingersea/3130 (Print.pdf, 91 slides). Coverage: all Ch1 knowledge points + worked Examples 1.1–1.14.